Mobile Application Development (3170726): Viva Notes

GTU Computer Engineering, Sem-7. Topics: Layouts, Manifest File, View, SharedPreferences.

1. Types of Layout

A layout defines how UI elements are arranged on screen. It is declared in XML (or built in code) and is a subclass of `ViewGroup`.

Layout	Description
LinearLayout	Arranges children in a single row (horizontal) or column (vertical). Uses `android:orientation` and `layout_weight`.
RelativeLayout	Positions children relative to each other or to the parent (e.g., `layout_below`, `layout_alignParentRight`).
ConstraintLayout	Positions views using constraints to siblings and the parent. Flat hierarchy, so better performance. Recommended today.
FrameLayout	Stacks children on top of each other. Used for a single view or as a fragment container.
TableLayout	Arranges children in rows and columns using `TableRow`.
GridLayout	Places children in a grid of rows and columns.
CoordinatorLayout	Advanced FrameLayout for Material Design behaviors (collapsing toolbar, FAB).

Example:

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView android:text="Hello" ... />
    <Button android:text="Click" ... />
</LinearLayout>
```

Follow-up they may ask: Why is ConstraintLayout preferred? It avoids deep nesting, which makes rendering faster.

2. Importance of the Manifest File

`AndroidManifest.xml` is a mandatory file in the root of every Android app. It tells the Android system everything it needs to know about the app before running any of its code.

Its roles:

1. Declares the package name, the app's unique identifier.
2. Declares app components: all Activities, Services, Broadcast Receivers, and Content Providers must be registered here.
3. Declares permissions the app needs (Internet, Camera, Location, etc.).
4. Defines the launcher activity using an intent filter with `MAIN` and `LAUNCHER`.
5. Specifies SDK requirements (min/target SDK, in older projects).
6. Declares hardware/software features the app requires (camera, GPS).
7. Defines intent filters that tell the system which implicit intents a component can handle.
8. Sets app-level attributes like icon, label, theme, and `allowBackup`.

Example:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">

    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="MyApp"
        android:theme="@style/AppTheme">

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

One-line answer: If a component isn't declared in the manifest, the system can't launch it, and the app will crash.

3. What is a View?

A View is the basic building block of the Android UI. It is a rectangular area on screen that is responsible for drawing itself and handling user interaction (touch, click, key events).

- The class is `android.view.View`.
- Every UI element is a subclass of View: `TextView`, `Button`, `EditText`, `ImageView`, `CheckBox`, etc.
- A ViewGroup is a special View that holds other Views (an invisible container). All layouts are ViewGroups.
- Views are arranged in a tree called the View Hierarchy.

Key difference:

View	ViewGroup
Single UI widget	Container for multiple views
Example: Button, TextView	Example: LinearLayout, ConstraintLayout

Common attributes: `id`, `layout_width`, `layout_height`, `padding`, `margin`, `visibility`, `background`.

Accessing a view in code:

```
TextView tv = findViewById(R.id.textView1);
tv.setText("Hello Android");
```

4. SharedPreferences

SharedPreferences is a mechanism to store small amounts of primitive data as key-value pairs. The data is saved in an XML file in the app's private storage and persists even after the app is closed.

Used for: login state, user settings, theme choice, "remember me" flag, first-launch flag.

Supported types: `boolean`, `int`, `float`, `long`, `String`, and `Set<String>`.

Getting a SharedPreferences object:

- `getSharedPreferences("name", MODE_PRIVATE)` for a named file.
- `getPreferences(MODE_PRIVATE)` for an activity-specific file.

Writing data:

```
SharedPreferences sp = getSharedPreferences("MyPrefs", MODE_PRIVATE);
SharedPreferences.Editor editor = sp.edit();
editor.putString("username", "Rahul");
editor.putBoolean("isLoggedIn", true);
editor.apply();   // or commit()
```

Reading data:

```
SharedPreferences sp = getSharedPreferences("MyPrefs", MODE_PRIVATE);
String name = sp.getString("username", "default");
boolean login = sp.getBoolean("isLoggedIn", false);
```

Clearing data: `editor.remove("username")` or `editor.clear()`, then `apply()`.

`apply()` vs `commit()`:

- `apply()` is asynchronous and returns nothing. Preferred.
- `commit()` is synchronous and returns a boolean for success.

Limitation: Not suitable for large or complex data. Use SQLite or Room instead.